Khang Nguyen

615-939-3645 | KhangN44345@gmail.com
github.com/KhangN65719 | linkedin.com/in/khangn445

EDUCATION

University of Illinois Chicago | Chicago, IL
Bachelor of Arts in Computer Science | Expected May 2027
Relevant coursework: Data Structures & Algorithms, Software Design, Machine Organization, Programming Language Concepts

TECHNICAL SKILLS

Languages: Java, Go (Golang), C++, Python, C, SQL
Tools & technologies: Git, GitHub, Docker, Railway, JavaFX, MySQL, SQLite, REST APIs, Maven
Testing: JUnit, Google Test (C++), unit testing

PROJECTS

Networked Checkers Application

Java, JavaFX, TCP Sockets | Apr 2026 | Team of 2

- Co-developed a Java client-server game with a multithreaded TCP socket server for player matchmaking, move relay, rematches, and disconnect handling.
- Implemented JavaFX gameplay with move validation, chained captures, king promotion, in-game chat, and a single-player computer opponent.

Personal Shell

C, POSIX System Calls

- Built a command-line shell that launches programs with fork and execv, resolves executables through PATH, and waits for child processes with waitpid.
- Implemented command pipelines using pipe and dup2, with input, output, and standard-error redirection through file descriptors.
- Added command sequencing and conditional execution with && and || based on command exit status.

GitHub Activity Bot

Go, REST APIs, Docker, Railway | Jun 2026

- Built a Go service that retrieves repository and commit activity from the GitHub REST API, decodes JSON responses, and delivers scheduled daily summaries through Gmail SMTP.
- Separated API access, report formatting, email delivery, and scheduling into dedicated packages; deployed to Railway with a multi-stage Docker build and Alpine runtime image.

Hospital Database Design

SQL, MySQL, Relational Modeling | Team of 3

- Designed a 16-table relational schema with primary, composite, and foreign keys in a three-person team to model patients, staff, treatments, room assignments, and billing.
- Created three SQL views for treatment information, staff workload, and billing balances; aggregated charges and payments separately before joining patient records to avoid duplicated totals.